305432 เครือข่ายเซนเซอร์ (Sensor Networks) · Assignment Paper 3

OmniLedger: A Secure, Scale-Out, Decentralized Ledger via Sharding

บัญชีแยกประเภทแบบกระจายศูนย์ที่ปลอดภัยและขยายขนาดแบบ Scale-Out ด้วยการแบ่ง Shard

2018 IEEE Symposium on Security and Privacy (SP) · San Francisco, CA, USA · หน้า 583–598
DOI: 10.1109/SP.2018.000-5 · IEEE Xplore: 8418625

Eleftherios Kokoris-Kogias, Philipp Jovanovic, Linus Gasser, Nicolas Gailly, Ewa Syta, Bryan Ford
École Polytechnique Fédérale de Lausanne (EPFL), Switzerland · Trinity College, USA

ผู้นำเสนอ: นายธีรภัทร ภู่ระย้า ลำดับที่ 10 · รหัสนิสิต 66362416

ที่มาและปัญหาความท้าทาย (Research Challenges & Problem Statement)

ความท้าทายหลัก 3 ประการที่ผู้เขียนตั้งไว้ (Section I)

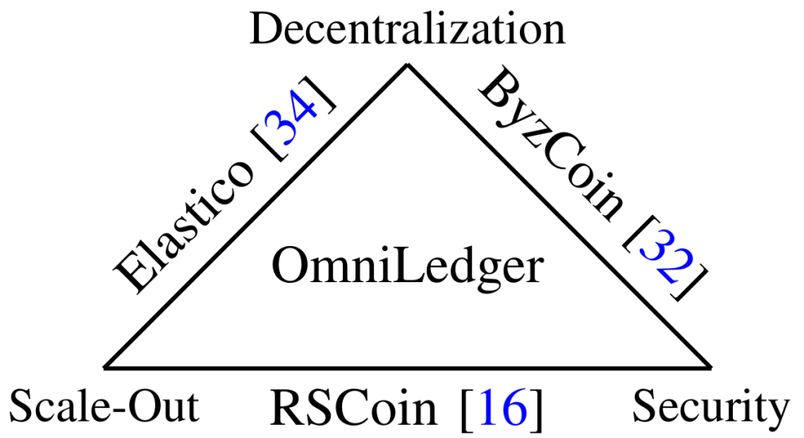
1 เลือก validator ที่เป็นตัวแทนทางสถิติ
สุ่มกลุ่ม validator จากระบบ permissionless โดยทนต่อ Sybil attack ผ่าน Proof-of-Work หรือ Proof-of-Stake

2 ไม่ให้ shard ใดถูกยึดตลอดอายุระบบ
shard ต้องใหญ่พอ และต้องสร้างใหม่เป็นระยะด้วยค่าสุ่มที่ผู้โจมตีบิดเบือนไม่ได้ (bias-resistant)

3 ธุรกรรมข้าม shard ต้องเป็น atomic
สำเร็จทั้งหมดหรือยกเลิกทั้งหมด เงินต้องไม่ถูกใช้ซ้ำและไม่ถูกล็อกค้างถาวร

แบบจำลองผู้โจมตี (Threat Model, Section III)

- validator ที่ไม่ซื่อสัตย์มีได้ไม่เกิน **25% (n = 4f)**
- ผู้โจมตีปรับตัวได้ช้าในระดับ epoch (mildly adaptive)
- ช่องทางสื่อสารระหว่าง validator ที่ซื่อสัตย์เป็นแบบ synchronous



ภาพที่ 1: ระบบเดิมได้เพียงสองในสามด้าน (Fig. 1 ของบทความ)

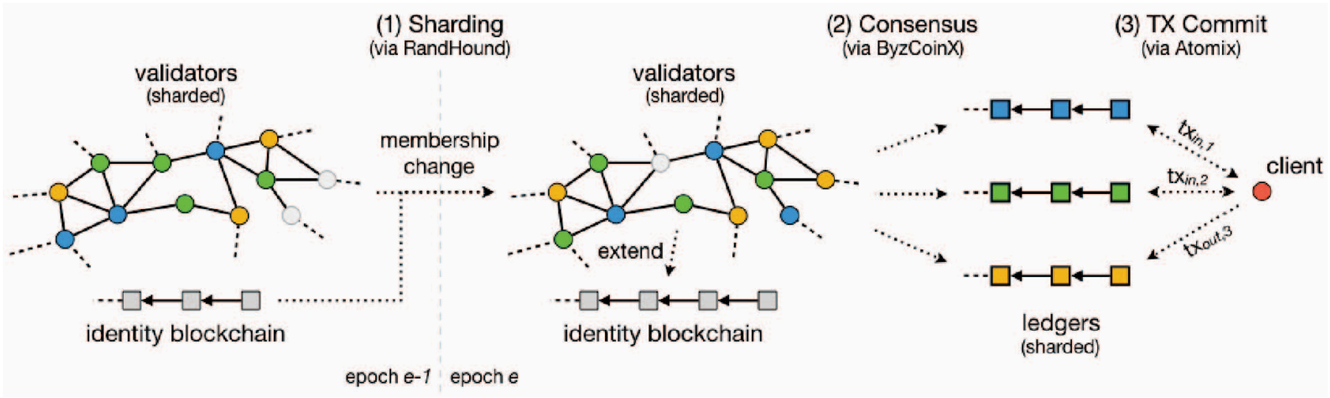
- **Elastico:** ขยายได้และกระจายศูนย์ แต่ shard เล็ก (100 validator) ล้มเหลว 2.76% ต่อ shard ต่อบล็อกที่ผู้โจมตี 25%
- **ByzCoin:** กระจายศูนย์และปลอดภัย แต่ความจุรวมไม่เพิ่มตามจำนวน validator
- **RSCoin:** ขยายได้และปลอดภัย แต่สละการกระจายศูนย์แบบ permissionless

1 ชม.
เวลาก่อนล้มเหลวของ Elastico (Table IV)

600 s
latency ของ Bitcoin (Table IV)

4,000
tx/s ค่าเฉลี่ยของ Visa ที่บทความใช้เทียบ

แบบจำลองเชิงตรรกะและตัวชี้วัด (Proposed Logical Model & Metrics)



ภาพที่ 2: สถาปัตยกรรม OmniLedger ในหนึ่ง epoch: (1) Sharding (2) Consensus (3) TX Commit (Fig. 2 ของบทความ)

1. Sharding ด้วยค่าสุ่มที่บิดเบือนไม่ได้

เลือกผู้นำด้วย VRF ให้ผู้นำรัน RandHound แล้วจัด validator เข้า shard ทยอยสลับครั้งละ k = log(n/m)

2. ByzCoinX ภายใน shard

Byzantine consensus สื่อสารแบบกลุ่มความลึกคงที่ ทนโหนดล้ม และ commit ธุรกรรมที่ไม่ขัดแย้งแบบขนาน

3. Atomix ข้าม shard

Initialize → Lock (proof-of-acceptance หรือ rejection) → Unlock to Commit หรือ Unlock to Abort

4. State blocks และ trust-but-verify

สรุปสถานะเพื่อตัดประวัติเก่า และให้กลุ่ม optimistic ยืนยันธุรกรรมมูลค่าต่ำก่อน core ตรวจซ้ำ

ตัวชี้วัดที่ใช้ประเมิน 4 ด้าน

Throughput (tps)
จำนวนธุรกรรมต่อวินาทีเมื่อเพิ่ม shard และสัดส่วนผู้โจมตี
Table II, Fig. 6

Confirmation latency (วินาที)
เวลายืนยันแบบ regular และแบบสองชั้นของ trust-but-verify
Table I, Fig. 8

Failure probability ต่อ epoch
โอกาสที่ shard ใดถูกยึด รวมทั้ง epoch ด้วย union bound
Eq. (1)–(2), Section VI

ต้นทุน bootstrap
เวลาเปลี่ยน epoch และแบนด์วิดท์ของ validator ใหม่
Fig. 7, Fig. 11

กรณีศึกษาและการจำลองระบบ (Scenario Study & System Formulation)

60 → 1,800
เครื่องจริงบน DeterLab → โหนดที่จำลอง (ต้นแบบภาษา Go)

20 Mbps · 100 ms
แบนด์วิดท์และความหน่วงที่กำหนดทุกลิงก์

10,000
บล็อกแรกของ Bitcoin ที่ใช้เป็นข้อมูลตั้งต้น

1% – 25%
สัดส่วนผู้โจมตีที่ทดสอบใน 4 การตั้งค่า

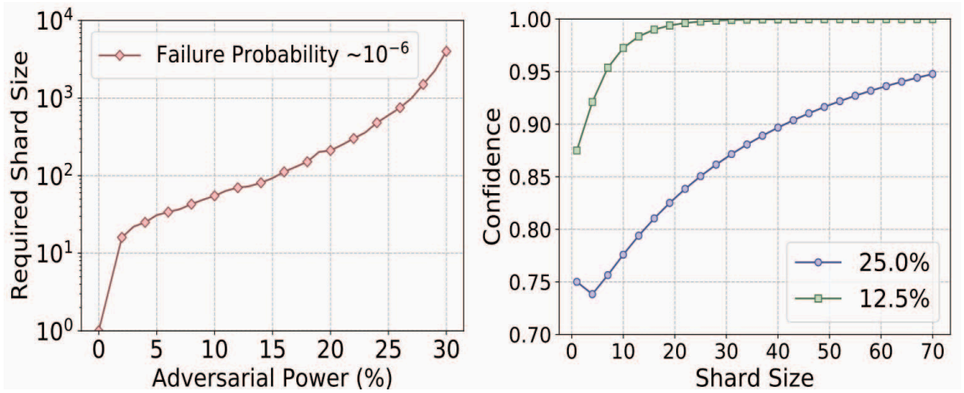
สมการหลักของความปลอดภัย (Section VI)

$$P[X \le \lfloor n/3 \rfloor] = \Sigma_{k=0}^{\lfloor n/3 \rfloor} C(n, k) \cdot m^k \cdot (1 - m)^{n-k} \quad (1)$$

$$P[X_E] \le \Sigma_{k=0}^{l} (1/4^k) \cdot n \cdot P[X_S] \Rightarrow (4/3) \cdot n \cdot P[X_S] \quad (2)$$

- **X** จำนวนโหนดไม่ซื่อสัตย์ใน shard, **m** สัดส่วนผู้โจมตี (≤ 0.25)
- **n** ขนาด shard ใน (1) แต่เป็นจำนวน shard ใน (2), **l** รอบที่ผู้โจมตีคุมผู้นำ
- **$P[X_S]$** โอกาส shard ล้มเหลว, **$P[X_E]$** โอกาสล้มเหลวทั้ง epoch
- ผู้โจมตี 12.5% กับ 16 shard ได้ **4×10^{-5} ต่อ epoch** (≈ 1 ครั้งใน 68.5 ปี)

หมายเหตุ: สมการ (1) เขียนตามรูปที่ถูกต้อง ในบทความพิมพ์ขอบบนของผลรวมเป็น n ซึ่งทำให้ผลรวมเท่ากับ 1 เสมอ (หน้า 591)



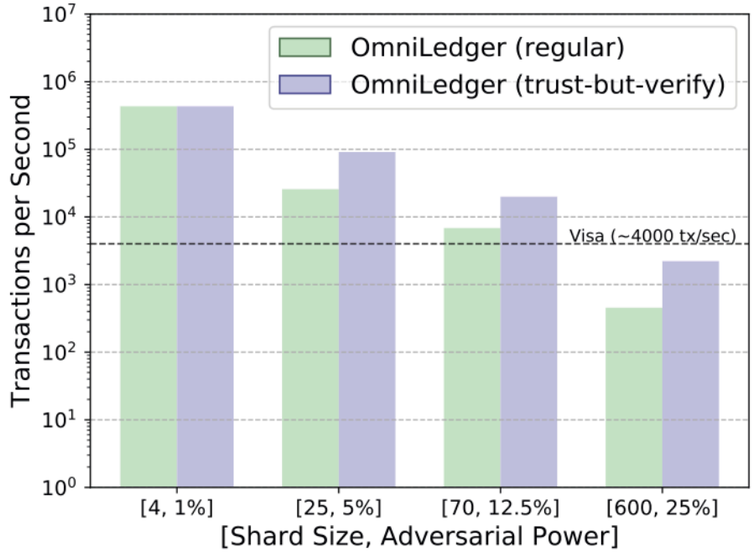
ภาพที่ 3: ขนาด shard ที่ต้องใช้เพื่อให้โอกาสล้มเหลว 10^{-6} (ซ้าย) และความมั่นใจของกลุ่ม optimistic (ขวา) (Fig. 5)

4 การตั้งค่าที่ทดสอบ [ขนาด shard, ผู้โจมตี]

[4, 1%] **[25, 5%]** **[70, 12.5%]** **[600, 25%]**

ผู้โจมตีมากขึ้น shard ต้องใหญ่ขึ้นมากเพื่อรักษาระดับความเสี่ยงเดิม

ผลการทดสอบเชิงปริมาณและข้อค้นพบ (Quantitative Evaluation & Findings)



ภาพที่ 4: throughput ที่ 1,800 โหนด ตามการตั้งค่า [ขนาด shard, ผู้โจมตี] (Fig. 6, สเกล log)

อ่านจากกราฟ (tps)	[4, 1%]	[25, 5%]	[70, 12.5%]	[600, 25%]
regular	431,600	25,700	6,800	450
trust-but-verify	431,600	91,200	19,900	2,200

ค่าในตารางผู้นำเสนออ่านจากพิกัดแท่งกราฟในไฟล์ PDF จึงเป็นค่าประมาณ บทความไม่ได้พิมพ์ตัวเลขเหล่านี้

439 → 5,850
tps เมื่อเพิ่ม shard จาก 1 เป็น 16 (Table II)

1.38 s
ยืนยันชั้นแรก (Table I) เมื่อผู้โจมตี ≤ 12.5%

4×10^{-5}
โอกาสล้มเหลวต่อ epoch ≈ 1 ครั้งใน 68.5 ปี

ข้อค้นพบสำคัญ

1. **Scale-out ใกล้เชิงเส้นแต่ลดลงตามจำนวน shard**
   ประสิทธิภาพต่อ shard 99.0% ที่ 2 shard เหลือ 83.3% ที่ 16 shard (Table II: shard ละ 70 โหนด ผู้โจมตี 12.5%)

2. **ByzCoinX ทำงานขนานได้ผล**
   throughput เพิ่ม 20% และ latency ต่อบล็อกลด 35% เมื่อแบ่งเป็น 4 บล็อกขนาน แต่ขนานมากกว่านั้นเริ่มช้าลง (Fig. 9)

3. **การเปลี่ยน epoch มีต้นทุนสูง**
   RandHound ใช้เวลามากกว่า 70% ของการ bootstrap กรณีแย่สุดประมาณ 3 ชั่วโมง ระบบจึงใช้ epoch ยาว 1 วัน (Fig. 7)

4. **ตัวเลขพาดหัวไม่ตรงกับกราฟ**
   Section VIII-B ระบุ **13,000 tps** ที่ 1,800 โหนด 25 shard แต่ Fig. 6 อ่านได้ ≈ 6,800 หรือ ≈ 19,900 tps และการยืนยันขั้นสุดท้ายที่ผู้โจมตี 25% ใช้ **62.96 s** (Table I)

คุณูปการทางวิชาการและการประยุกต์ใช้ (Academic Contributions & Practical Impact)

คุณูปการทางวิชาการ (Academic Contributions)

1. การแบ่ง shard ที่วัดความเสี่ยงได้

RandHound ร่วมกับ VRF สุ่มสมาชิก shard ที่ผู้โจมตีบิดเบือนไม่ได้ และ Eq. (1)–(2) ผูกขนาด shard กับโอกาสล้มเหลว (เช่น 10^{-6})

หลักฐาน: Fig. 5 และ Table IV: เวลาก่อนล้มเหลว 68.5 ปี เทียบ Elastico ที่ใช้ shard เล็ก 1 ชั่วโมง

2. Atomix: atomic commit ข้าม shard

lock แล้ว unlock โดยใช้ proof ที่มีลายเซ็นของ shard เป็นหลักฐาน ไม่ต้องมีผู้ประสานกลางและ shard ไม่ต้องคุยกันเอง

หลักฐาน: Fig. 8: shard 70 โหนด latency ที่ client เห็นราว 9 ถึง 12 s จาก 1 ถึง 7 shard (shard 600 โหนดเพิ่มจากราว 15 เป็น 26 s)

3. วิศวกรรมระบบที่วัดผลได้

ByzCoinX, state blocks และ trust-but-verify ถูกพัฒนาเป็นต้นแบบภาษา Go และทดลองที่ 1,800 โหนด

หลักฐาน: ByzCoinX +20% / −35% (Fig. 9) และ state blocks คุ้มเมื่อ validator ตามไม่ทันเกิน 19 วัน (Fig. 11)

การประยุกต์ใช้ (ความเห็นของผู้นำเสนอ)

1. Kafka partition และข้อความข้าม partition

shard คล้าย partition ที่กระจายตาม key จึงขยาย throughput ด้วยการแบ่ง state เหมือนกัน

จุดต่าง: Kafka รับมือโหนดล้ม (crash fault) แต่ไม่ได้ออกแบบให้ทนโหนดที่ตั้งใจโกง จึงไม่ต้องสุ่มสมาชิกใหม่ทุก epoch

2. Two-phase commit และ Saga ใน microservices

Atomix มีโครง lock แล้ว commit หรือย้อนกลับ คล้าย prepare/commit ของ two-phase commit

จุดต่าง: Atomix ใช้ client และ proof ที่มีลายเซ็นแทน coordinator ส่วน Saga ย้อนงานด้วย compensating transaction

3. ชั้น gateway ของเครือข่ายเซนเซอร์

แบ่ง gateway ตามพื้นที่เป็นกลุ่ม สุ่มสลับสมาชิก และบันทึกเหตุการณ์ข้ามพื้นที่แบบ atomic

ข้อจำกัด: ไม่เหมาะรันบนโหนดเซนเซอร์ เพราะ BFT และลายเซ็นดิจิทัลมีต้นทุนพลังงานและการสื่อสารสูง

ข้อจำกัดทางวิศวกรรมและบทสรุป (Engineering Limitations & Conclusion)

ข้อจำกัดที่ตรวจพบ (Limitations)

1. ผล scale-out ไม่ได้แยกผลของธุรกรรมข้าม shard



ประเด็น: ค่าอุดมคติคือ 4× บทความไม่ระบุสัดส่วนธุรกรรมข้าม shard ในการทดลอง throughput ขณะที่ Appendix C คาดว่าธุรกรรมส่วนใหญ่จะข้าม shard

2. สมการและตัวเลขไม่สอดคล้อง

$$P(n,k,m) = \begin{cases} 1, n = 1, k = 1 \\ 0, n = 1, k \neq 1 \\ \frac{m-k}{m} P(n-1,k-1,m) + \\ \frac{k}{m} P(n-1,k,m), n \neq 1, k > 0 \end{cases} \quad (3)$$

Eq. (3) ตามที่พิมพ์ (Appendix C, หน้า 597): สัมประสิทธิ์ (m − k)/m ควรเป็น (m − k + 1)/m

จุดอื่น: Eq. (1) ขอบบนเป็น n, throughput มีทั้ง 6,000 และ 13,000 tps, Table IV อ้าง 2,400 validator ทั้งที่ทดลองสูงสุด 1,800 โหนด

3. ขอบเขตการทดลองจำกัด

เครือข่ายจำลองแบบค่าคงที่ กรณีผู้โจมตี 25% ที่ 4,000 tps (≈ 4,200 โหนด) เป็นเพียงค่าประมาณ และไม่รายงานส่วนเบี่ยงเบนมาตรฐาน

บทสรุปและการต่อยอด (Conclusion)

1. ข้อสรุปหลัก

sharding ร่วมกับ BFT consensus และ atomic commit ทำให้ขยายแบบ scale-out ได้ โดยวัดความเสี่ยงเป็นตัวเลข

หลักฐาน: 439 → 5,850 tps (Table II), ยืนยันชั้นแรก 1.38 s และ 4×10^{-5} ต่อ epoch

2. วิธีนำไปใช้อย่างระมัดระวัง

นำแนวคิดสถาปัตยกรรมและแบบจำลองความเสี่ยงไปใช้ และทดสอบ throughput กับภาระงานจริงก่อนอ้างอิงตัวเลข

ตัวอย่าง: วัดสัดส่วนธุรกรรมที่ข้าม partition ก่อนเลือกจำนวน shard

3. งานต่อยอดที่ผู้เขียนระบุ

จัด shard ตาม locality เพื่อลดธุรกรรมข้าม shard ลดต้นทุน epoch ด้วย BLS signature และเพิ่มการป้องกัน censorship

ที่มา: Section X (Limitation and Future Work) และ Appendix A